

College Electrical Engineering Short

Complete Concise Lecture Notes Review & Core Formulas

PAGE 1

1.1 Electromagnetics & Maxwell's Equations

Maxwell's equations describe how electric and magnetic fields propagate and interact with matter. In differential form:

- **Faraday's Law of Induction:**

$$\nabla \times \vec{E} = - \frac{\partial \vec{B}}{\partial t}$$

- **Ampere's Circuital Law:**

$$\nabla \times \vec{H} = \vec{J}_f + \frac{\partial \vec{D}}{\partial t}$$

- **Gauss's Law for Electricity:**

$$\nabla \cdot \vec{D} = \rho_f$$

- **Gauss's Law for Magnetism:**

$$\nabla \cdot \vec{B} = 0$$

1.2 Electromagnetic Potentials

To solve Maxwell's equations, we define vector potential $\vec{A}$ and scalar potential ϕ :

- **Magnetic Vector Potential:**

$$\vec{B} = \nabla \times \vec{A}$$

- Electric Field relation:

$$\vec{E} = -\nabla \phi - \frac{\partial \vec{A}}{\partial t}$$

• Constitutive Relations:

$$\vec{B} = \mu \vec{H}, \quad \vec{D} = \epsilon \vec{E}, \quad \vec{J}_c = \sigma \vec{E}$$

1.3 Poynting's Theorem & Wave Equations

- Poynting's Theorem (Power Flow):

$$-\nabla \cdot \vec{S} = \vec{J}_c \cdot \vec{E} + \vec{E} \cdot \frac{\partial \vec{D}}{\partial t} + \vec{H} \cdot \frac{\partial \vec{B}}{\partial t}$$

where $\vec{S} = \vec{E} \times \vec{H}$ is the Poynting vector representing power density.

- **Wave Equations under Lorentz Gauge** ($\nabla \cdot \vec{A} = -\mu\epsilon \frac{\partial \phi}{\partial t}$):
- For Vector Potential:

$$\nabla^2 \vec{A} - \mu\epsilon \frac{\partial^2 \vec{A}}{\partial t^2} = -\mu \vec{J}_f$$

- For Scalar Potential:

$$\nabla^2 \phi - \mu\epsilon \frac{\partial^2 \phi}{\partial t^2} = -\frac{\rho_f}{\epsilon}$$

2.1 Systems Analysis Domains

Signal processing systems can be analyzed in different mathematical domains depending on time continuity:

- **Continuous-Time Domain (t-domain):**
- Input-output relation is defined by convolution:

$$y(t) = x(t) * h(t) = \int_{-\infty}^{\infty} x(\tau)h(t - \tau)d\tau$$

- **Laplace Domain (s-domain):**

- Transform definition:

$$X(s) = \int_0^{\infty} x(t)e^{-st}dt$$

- System output:

$$Y(s) = X(s)H(s)$$

- **Discrete-Time Domain (n-domain):**

- Input-output relation:

$$y(n) = x(n) * h(n) = \sum_{m=-\infty}^{\infty} x(m)h(n - m)$$

- **Z-Domain (z-domain):**
- Transform definition:

$$X(z) = \sum_{n=-\infty}^{\infty} x(n)z^{-n}$$

- System output:

$$Y(z) = X(z)H(z)$$

2.2 Discrete Fourier Transform & FFT

- **DFT Definition:**

$$X(k) = \sum_{n=0}^{N-1} x(n)e^{-j\frac{2\pi}{N}kn}$$

- **Decimation-in-Time (DIT) Radix-2 FFT:** Splits the input sequence into even and odd indices, reducing complexity from $O(N^2)$ to $O(N \log N)$.

- **Decimation-in-Frequency (DIF) Radix-2 FFT:** Splits the output spectrum into even and odd frequencies.

- **Split-Radix FFT:** Combines Radix-2 and Radix-4 algorithms to achieve a minimum number of complex multiplications.

2.3 Digital Filter Structures

Digital filters are realized in hardware or software using basic structural building blocks:

- **Direct Form (DF-I & DF-II):** Direct implementation of the difference equation. DF-II minimizes memory registers.

- **Cascade Form:** Realizes the transfer function as a product of second-order sections (biquads).

- **Parallel Form:** Realizes the transfer function as a sum of second-order sections via partial fraction expansion.

3.1 Digital Filter Design Methods

- **Impulse Invariant Method:**
- Characteristics: Maps analog poles to digital poles. Aliasing occurs if the analog frequency response is not band-limited.
- Formula:

$$H(e^{j\omega}) = \frac{1}{T} \sum_{n=-\infty}^{\infty} H_a\left(j\frac{\omega}{T} + j\frac{2\pi}{T}n\right)$$

- Pole mapping:

$$\frac{1}{s - s_n} \rightarrow \frac{1}{1 - e^{s_n T} z^{-1}}$$

- **Bilinear Transform Method:**
- Characteristics: Maps the entire analog frequency range to the digital range. No aliasing occurs.
- Distortion: Frequency warping occurs.

3.2 Ergodicity & Karhunen-Loève (K-L) Expansion

- **Ergodicity:** A stationary random process is ergodic if its time average equals its ensemble average.
- **K-L Expansion:** Expands a random process $X(t)$ using the eigenfunctions of its autocorrelation function $R_X(t, s)$ as an orthonormal basis. The expansion coefficients are uncorrelated.

3.3 Quantization Noise & Filter Structure

- **Scaling designs:** Low-sensitivity realization using cascade/parallel structures with second-order sections.
- **Quantization Noise:**
- Quantization error variance:

$$\sigma_q^2 = \frac{\Delta^2}{12}$$

- Single noise source output variance:

$$\sigma_{no}^2 = \sigma_q^2 \sum_{n=-\infty}^{\infty} h(n)^2 = \sigma_q^2 \frac{1}{2\pi} \int_{-\pi}^{\pi} |H(e^{j\omega})|^2 d\omega$$

- Multi-source output noise variance sum:

$$\sigma_{total}^2 = \sum_m \Delta^2 \sum h_m(n)^2$$

- Scaling factor: $\beta < 1$ is used to prevent overflow.

3.4 Random Processes & Stationarity

- **Envelope Noise:**

- Broadband Gaussian noise without sine wave: Rayleigh distribution.

- Broadband Gaussian noise with sine wave: Rice distribution.

- **Second-Order Process:** A process whose mean $E[X(t)]$ and variance $D[X(t)]$ exist.

- **Strictly Stationary Process (SSS):** Joint distribution is invariant under time shifts.

- **Weakly Stationary Process (WSS):**

- Mean is constant: $E[X(t)] = m_X$

- Autocorrelation depends only on time difference: $R_X(t, s) = R_X(t - s)$

- Poisson Process:

- Probability distribution:

$$P(N(t) = n) = \frac{(\lambda t)^n}{n!} e^{-\lambda t}$$

- Has independent and stationary increments.

- **Compound Poisson Process:**

$$X(t) = \sum_{k=1}^{N(t)} Y_k$$

4.1 Markov Chain Classifications

- **Markov Property:**

$$P(X_{n+1} = j | X_n = i, X_{n-1} = i_{n-1}, \dots, X_0 = i_0) = P(X_{n+1} = j | X_n = i)$$

- **State Classification:**

- **Transient State (Non-recurrent):** $f_{ii} < 1$, and $\sum_{n=1}^{\infty} P_{ii}^{(n)} < \infty$.

- **Recurrent State:** $f_{ii} = 1$, and $\sum_{n=1}^{\infty} P_{ii}^{(n)} = \infty$.

- **Positive Recurrent State:** Recurrent state with finite mean recurrence time $\mu_i < \infty$.

- **Null Recurrent State:** Recurrent state with infinite mean recurrence time $\mu_i = \infty$.

- **Stationary Distribution:** A probability distribution π satisfying $\pi = \pi P$.

4.2 Analog Electronics: Amplifier Models

- **DC Model:** Used to determine the quiescent operating point (Q-point).

- **AC Model:** Used to analyze the small-signal AC response (AC load lines).

- **Mid-band Small-Signal Model:** Capacitors are treated as short circuits or open circuits. High frequency parasitic parameters are ignored. Determines mid-band input resistance R_{in} , output resistance R_{out} , and voltage gain A_v .

- **High-Frequency Small-Signal Model (Hybrid- π Model):** Includes BJT internal capacitances (C_π, C_μ) and base-spreading resistance ($r_{bb'}$). Hybrid- π elements:

- Base-spreading resistance: $r_{bb'}$

- Base-to-internal-emitter resistance: $r_{b'e}$

- Diffusion/depletion capacitance: C_π (or C_e)

- Collector depletion capacitance: C_{μ} (or C_c)

- Transconductance: $g_m V_{b'e}$

5.1 Operational Amplifiers (Op-Amps)

- **Characteristics:** High input impedance, low output impedance, and high open-loop gain.
- **Linear Applications:** Summing amplifier, subtractor, differentiator, integrator (requires negative feedback for stability).
- **Comparators:** Open-loop configuration or with positive feedback to create hysteresis (Schmitt trigger).
- **Voltage Follower:** Unity-gain buffer using direct negative feedback.

5.2 ADC & DAC Architectures

- **Digital-to-Analog Converters (DAC):**
- **Weighted Resistor DAC:** Combines binary-weighted currents.
- **R-2R Ladder DAC:** Uses only two resistor values (R and 2R) to scale current.
- **Analog-to-Digital Converters (ADC):**
- **Flash ADC (Parallel comparator):** Fastest conversion speed; requires $2^n - 1$ comparators.
- **Successive Approximation Register (SAR) ADC:** Uses binary search via feedback DAC; medium speed, low cost.
- **Dual-Slope Integrating ADC:** High accuracy, slow speed, high noise immunity.

5.3 Digital Logic & Memory

- **Logic Gates:** Implemented using BJTs (TTL) or MOSFETs (CMOS) to perform bit-by-bit digital processing.
- **Decoder:** Converts an n-bit input to 2^n unique outputs.

- **Encoder:** Converts 2^n input lines to an n-bit binary code.
- **Multiplexer (Selector):** Selects one of many input signals to route to a single output.
- **Demultiplexer (Distributor):** Routes a single input to one of many outputs.
- **Adders:** Ripple carry adder vs. Carry lookahead adder (eliminates carry hazards).
- **Microprocessor Byte Order:**
- **Little-Endian:** Low address stores the least significant byte (LSB) (e.g., x86).
- **Big-Endian:** Low address stores the most significant byte (MSB) (e.g., Network protocols).

6.1 Computer Architecture & ISA

- **Software Levels:**
- **Assembler:** Translates assembly instructions (e.g., MASM) to machine code.
- **Compiler:** Translates high-level language (e.g., C/C++) to machine code.
- **Interpreter:** Directly executes scripts (e.g., MATLAB) without compilation.
- **CPU Time Formula:**

$$\text{textCPUTime} = \text{textInstructionCount} \times \text{textCPI} \times \text{textClockCycleTime}$$

where textCPI is Clock Cycles Per Instruction.

- Architectures:

- **CISC (Complex Instruction Set Computer):** Fewer instructions, higher CPI, complex hardware.
- **RISC (Reduced Instruction Set Computer):** More instructions, lower CPI (aims for 1), simple hardware.
- **Instruction Set Architecture (ISA) types:** Stack, Accumulator, Register-Memory, Register-Register (Load-Store).

6.2 x86 Real Mode Memory Segmentation

- **Memory limit:** 1 MB address space, 20-bit address bus.
- **Address translation:** 16-bit segment address shifted left by 4 bits plus a 16-bit offset:

$$\text{textPhysicalAddress} = \text{textSegment} \times 16 + \text{textOffset}$$

- **Segment Registers:**
- **`CS` (Code Segment):** Points to instruction memory (with IP).

- `DS` (Data Segment): Points to data memory (with BX, SI, DI).
- `SS` (Stack Segment): Points to stack memory (with SP, BP).
- `ES` (Extra Segment): Points to destination strings (with DI).

6.3 CPU Pipelining & CPU Hazards

- **Control Methods:** Hardwired control (fast) vs. Microprogrammed control (flexible but slower).
- **Pipelining Hazards:**
- **Structural Hazards:** Resource conflict where hardware cannot support combination of instructions.
- **Data Hazards:** Instruction depends on the result of a previous instruction still in the pipeline.
- **Control Hazards:** Branch instructions determine the next instruction to fetch.

7.1 C++ Variables and Storage Classes

- ``static``: Local static variables retain their values between function calls and are stored in the data segment. Global static variables have file scope.
- ``const``: Variables declared `const` are immutable and must be initialized upon declaration.

7.2 Object-Oriented Programming (OOP) in C++

- **Polymorphism:**
- **Static Polymorphism (Compile-time):** Overloading (functions/operators) and templates (class/function templates).
- **Dynamic Polymorphism (Run-time):** Inheritance and virtual functions.
- **Virtual Functions:** Declared with ``virtual`` keyword. Checked at run-time (via V-table) to invoke the correct derived function.
- **Constructors/Destructors:** Constructors initialize data members and bind memory. Destructors can be virtual to ensure clean deletion of derived objects.
- **Friend Classes/Functions:** Can access private and protected members of another class. Friends are one-way and are not inherited.
- **I/O Streams:** ``ofstream`` (inherits from ``ostream`` for output files) and ``ifstream`` (inherits from ``istream`` for input files).

7.3 Numerical Methods

- **Root Finding:**
- **Bisection Method:** Divides the interval containing the root in half iteratively.
- **Newton-Raphson Method:** Fast iterative tangent-line method:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

- Ordinary Differential Equations (ODEs):

- Euler's Method: Iteratively steps forward:

$$y_{n+1} = y_n + hf(t_n, y_n)$$

- **Linear Systems:** Iterative solvers such as Jacobi iteration and Gauss-Seidel iteration. Converges if spectral radius of iteration matrix $\rho(B) < 1$.
- **Regression:** Solving overdetermined systems by minimizing the sum of squared residuals (Least Squares).

8.1 String Matching Algorithms

- **KMP (Knuth-Morris-Pratt) Algorithm:**
- Uses a precomputed `next` array representing prefix matching lengths to skip redundant character comparisons.
- Time complexity: $O(m + n)$.

8.2 Sorting Algorithms Summary

- **Bubble Sort:** Compares adjacent elements and swaps them. Stable, $O(n^2)$.
- **Quicksort:** Divides array using a pivot. Unstable, average $O(n \log n)$, worst-case $O(n^2)$.
- **Direct Insertion Sort:** Inserts each item into its correct place in the sorted prefix. Stable, $O(n^2)$.
- **Binary Insertion Sort:** Uses binary search to find the insertion index. Stable, $O(n^2)$.
- **Shell Sort:** Sorts sub-arrays separated by a diminishing gap. Unstable, $O(n^{1.5})$.
- **Simple Selection Sort:** Finds the minimum element and swaps it. Unstable, $O(n^2)$.
- **Merge Sort:** Recursively divides and merges sorted halves. Stable, $O(n \log n)$.
- **Heap Sort:** Builds a heap and repeatedly extracts the root. Unstable, $O(n \log n)$.

8.3 RF Impedance Matching & Power Amplifiers

- **Maximum Power Transfer:** For complex source impedance Z_s , maximum power is delivered to the load when:

$$Z_L = Z_s^*$$

- **Conjugate Matching:** Match transmission line impedance Z_c with load Z_L .
- **Resonant Matching Networks:** L-networks, π -networks, and T-networks are used to transform impedances and filter harmonics.
- **Power Amplifier Classes:**
 - **Class A:** Conduction angle 360° (always conducting), low efficiency ($\eta < 50\%$), highly linear.
 - **Class B:** Conduction angle 180° , max efficiency $\eta \approx 78.5\%$, uses push-pull configuration to reduce distortion.
 - **Class C:** Conduction angle $< 180^\circ$, high efficiency, non-linear, requires resonant LC tank load.
 - **Class D:** Switching amplifier, theoretical efficiency up to 100%.
- **Oscillators (Barkhausen Criteria):**
 - **Amplitude Condition:** $|AF| = 1$ (startup requires $|AF| > 1$).
 - **Phase Condition:** $\phi_A + \phi_F = 2n\pi$.

9.1 RF Oscillators

- **LC Oscillators:**
- **Colpitts Oscillator:** Uses a capacitive divider in the resonant tank feedback.
- **Hartley Oscillator:** Uses an inductive divider in the resonant tank feedback.
- **Clapp / Seiler Oscillators:** Modified Colpitts with an extra capacitor in series (Clapp) or parallel (Seiler) to improve stability.
- **Active Stage:** Common-Emitter (CE) and Common-Base (CB) are suitable; Common-Collector (CC) is not suitable due to lack of voltage gain.
- **Crystal Oscillators (Quartz):**
- **Series Mode:** Crystal acts as a low-impedance short circuit at series resonance f_s .
- **Parallel Mode:** Crystal acts as a high-value inductor between f_s and f_p (e.g., Pierce oscillator).

9.2 Superheterodyne Receivers

- **Architecture:** Antenna → LNA → RF Filter → Mixer (with LO) → IF Filter → Demodulator.
- **Zero-IF (Direct Conversion):** Down-converts RF directly to baseband. Eliminates image frequency filter requirements but is sensitive to DC offset.
- **Image Rejection Mixers:** Hartley and Weaver architectures reject the image frequency using phase shifting.

9.3 FM Modulation & Demodulation

- **Direct FM:** A varactor diode directly modulates the frequency of an LC tank oscillator.

- **Indirect FM (Armstrong Method):** Integrates the message first, performs phase modulation (PM), and then uses frequency multipliers to increase frequency deviation.
- **FM Demodulation:**
- **Slope Detector:** Double-tuned circuits convert FM to AM (differentiation), followed by envelope detection.
- **Quadrature Detector (Delay Line):** Multiplies the FM signal with a phase-shifted version of itself.
- **Phase-Locked Loop (PLL) Demodulator:** VCO output tracks input FM frequency; loop filter voltage is the demodulated output.

10.1 Electromagnetic Waveguides

Waveguides guide high-frequency electromagnetic waves using reflective boundary conditions:

- **Guided Wave Equations:**

- Guided wave number:

$$k_z = \sqrt{k^2 - k_c^2}$$

- Guided wave wavelength:

$$\lambda_g = \frac{2\pi}{k_z}$$

- Cutoff frequency:

$$f_c = \frac{v}{2\pi} k_c$$

- Cutoff wavelength:

$$\lambda_c = \frac{2\pi}{k_c}$$

- **Field Decomposition:** Fields decompose into transverse electric (TE, $E_z = 0$) and transverse magnetic (TM, $H_z = 0$) modes.

10.2 Transmission Line Theory

- **Telegrapher's Equations:**

$$\frac{dV(z)}{dz} = -(R + j\omega L)I(z)$$

$$\frac{dI(z)}{dz} = -(G + j\omega C)V(z)$$

- **Line Terminations:**
- **Open Circuit:** $Z_L = \infty$, Reflection Coefficient $\Gamma_L = 1$, SWR $\rho = \infty$.
- **Short Circuit:** $Z_L = 0$, Reflection Coefficient $\Gamma_L = -1$, SWR $\rho = \infty$.
- **Pure Reactance:** $Z_L = jX$, $|\Gamma_L| = 1$, SWR $\rho = \infty$.
- **Matched Load:** $Z_L = Z_c$, Reflection Coefficient $\Gamma_L = 0$, SWR $\rho = 1$ (pure traveling wave).

10.3 Microwave Networks & Antennas

- **Scattering Parameters (S-parameters):** Relates incident power waves (a) to reflected waves (b):

$$\mathbf{b} = [\mathbf{S}] \mathbf{a}$$

For a matched port i , $S_{ii} = 0$.

- **Antennas:** Wire antennas, aperture antennas, and microstrip patch antennas. Antenna arrays use the pattern multiplication theorem:

$$\text{Total Pattern} = \text{Element Factor} \times \text{Array Factor}$$

10.4 Information Theory

- **Discrete Source Entropy:** Measures source uncertainty:

$$H(X) = - \sum_{i=1}^M P(x_i) \log_2 P(x_i)$$

- **Conditional Entropy:**

$$H(Y|X) = - \sum_j P(y_j|x_i) \log_2 P(y_j|x_i)$$

- **Channel Capacity:**

$$C = \max H(X) - H(X|Y) = H(Y) - H(Y|X)$$

- Continuous Source Entropy:

$$H(X) = - \int_{-\infty}^{\infty} p(x) \log_2 p(x) dx \leq \log_2(\sqrt{2\pi} \sigma_X)$$

- **Shannon-Hartley Theorem:**

$$C = B \log_2 \left(1 + \frac{S}{N} \right)$$

11.1 Channel Coding (Error Correction)

- **Hamming Codes:** $(2^r - 1, 2^r - 1 - r)$ block codes with minimum Hamming distance $d_{\min} = 3$. Can correct 1-bit errors.
- **Linear Block Codes:**
 - Generator matrix $G = [I_k \mid P]$. Parity check matrix $H = [-P^T \mid I_{n-k}]$.
 - Encoding: $C = M \cdot G$.
 - Syndrome: $S = R \cdot H^T$ (if $S = 0$, no errors detected).
- **Cyclic Codes:** Realized using feedback shift registers:
- Codeword polynomial:

$$C(X) = X^{n-k}M(X) + (X^{n-k}M(X) \bmod g(X))$$

- Parity check:

$$S(X) = R(X) \bmod g(X)$$

- **Convolutional Codes:** Realized by passing data through shift registers. Decoded using the Viterbi algorithm over a trellis diagram.

11.2 Digital Communication Systems

- **Baseband Equalization:** Duobinary signaling (partial response) introduces controlled ISI to achieve Nyquist rate without brick-wall filters.
- **Modulation Schemes:** Minimum Shift Keying (MSK) features a constant envelope and continuous phase, minimizing spectral side-lobes.
- **Optimal Receiver:** Matched filters maximize output SNR in the presence of AWGN:

$$h(t) = s(T - t)$$

$$\text{PeakSNR} = \text{PeakSignalEnergy} / (N_0 / 2)$$

12.1 Graph Algorithms

- **Minimum Spanning Tree (MST):**
- **Prim's Algorithm:** Starts from a root node and adds the closest unvisited node. $O(V^2)$ or $O(E \log V)$, preferred for dense graphs.
- **Kruskal's Algorithm:** Sorts edges by weight and adds them if they don't form a cycle. $O(E \log E)$, preferred for sparse graphs.
- **Shortest Paths:**
- **Dijkstra's Algorithm:** Solves single-source shortest path for non-negative edge weights.
- **Floyd-Warshall Algorithm:** Solves all-pairs shortest paths using dynamic programming: $O(V^3)$.
- **Critical Path Method (CPM):** Analyzes AOE (Activity On Edge) networks using topological sorting to find the earliest/latest event occurrence times.

12.2 Searching & Indexing

- **Search Techniques:**
- **Sequential Search:** Scan elements one-by-one. $O(n)$.
- **Binary Search:** Divide-and-conquer on sorted lists. $O(\log n)$.
- **Binary Search Tree (BST):** In-order traversal yields sorted elements. Can be self-balanced (AVL trees) to ensure $O(\log n)$ performance.
- **B-Trees:** Balanced m-way search trees designed for disk indexing.
- **KMP Pattern Matching:** Utilizes a failure function `next` to avoid back-tracking on the text string.

12.3 Embedded System Design

Case study: Digital Temperature Monitor

- **Calibration:** Multi-point calibration to correct non-linearity.
- **Analog-to-Digital Conversion:** Periodic reading updates using a built-in ADC.
- **Temperature Compensation:** High-temperature and low-temperature range compensations.
- **Over-temperature Action:** Log maximum peak temperatures and trigger alarms.